

Agent Console 控制台需求文档

1. 项目背景

公司内部需要一个用于管理多个项目、多 Agent 协作、实时对话和终端执行状态查看的统一控制台。

当前 AI Agent 使用场景中，通常会同时存在多个项目，每个项目下有多个 Agent，例如开发 Agent、测试 Agent、部署 Agent、数据分析 Agent 等。使用者需要在在一个页面中完成以下操作：

1. 选择项目与 Agent；
2. 与指定 Agent 对话；
3. 查看该 Agent 对应的 Node TTY / Terminal 执行状态；
4. 快速发送常用命令，例如 `/clear`、`/compact`、`/help`；
5. 对项目、Agent、常用命令进行增删改管理。

本系统目标是提供一个三列式 Web 控制台，提高 AI Agent 使用效率和管理清晰度。

2. 产品定位

产品名称暂定：**Agent Console**

产品形态：Web 应用

主要用户：

- 技术负责人
- 程序开发人员
- 测试人员
- 运维人员
- AI Agent 使用者

核心价值：

- 将「项目管理」「Agent 对话」「Terminal 执行状态」集中在一个页面；
- 降低多窗口切换成本；
- 支持多个项目和多个 Agent 的统一管理；
- 支持常用命令快捷发送，提高操作效率；
- 为后续接入自动化开发、部署、测试、日志分析打基础。

3. 页面整体布局

页面采用三列布局：

项目/Agent菜单	人与Agent对话区	Node TTY Terminal区
------------	------------	--------------------

项目列表	消息列表	实时终端输出	
Agent列表	常用命令Tag	命令执行日志	
新增/删除	输入框	状态信息	

3.1 第一列：项目 / Agent 菜单区

左侧第一列用于展示项目和 Agent 的树形结构。

一级菜单代表项目，二级菜单代表该项目下的 Agent。

示例：

```
E-Commerce
- OrderAgent
- InventoryAgent
- PaymentAgent

Data Analytics
- QueryAgent
- ChartAgent

DevOps
- DeployAgent
- MonitorAgent
```

3.2 第二列：对话区

中间第二列用于展示人与当前选中 Agent 的对话内容。

主要包含：

- 当前项目名称；
- 当前 Agent 名称；
- Agent 在线状态；
- 历史消息列表；
- 常用命令 Tag 区；
- 消息输入框；
- 发送按钮；
- 附件、图片、代码等快捷入口。

3.3 第三列：Node TTY Terminal 区

右侧第三列用于展示当前 Agent 对应的 Node TTY 模拟终端。

主要包含：

- Terminal 标题；

- 当前运行状态；
- 命令输出；
- 日志输出；
- 进程状态；
- 清屏、复制、放大、关闭等操作按钮。

4. 功能需求

4.1 项目管理

4.1.1 项目列表展示

系统需要在左侧菜单中展示所有项目。

每个项目展示内容包括：

- 项目名称；
- 项目图标；
- 展开 / 收起状态；
- Agent 数量；
- 操作按钮。

4.1.2 新增项目

用户可以点击「新建项目」按钮创建项目。

新增项目字段：

字段	必填	说明
项目名称	是	项目的显示名称
项目标识	是	系统内部唯一 key
项目描述	否	项目说明
项目图标	否	可选图标
默认 Agent	否	创建项目时可同时创建默认 Agent

交互要求：

- 点击「新建项目」后弹出创建弹窗；
- 填写完成后点击确认；
- 创建成功后项目出现在左侧菜单；
- 新项目默认展开。

4.1.3 编辑项目

用户可以编辑项目基础信息。

可编辑字段：

- 项目名称；
- 项目描述；
- 项目图标；
- 项目排序。

4.1.4 删除项目

用户可以删除项目。

删除规则：

- 如果项目下存在 Agent，需要二次确认；
- 删除项目会同时删除或归档该项目下的 Agent；
- 删除前需要提示用户该操作不可逆，或者进入回收站。

建议支持两种删除方式：

1. 软删除：项目进入回收站；
2. 硬删除：管理员才可以彻底删除。

4.2 Agent 管理

4.2.1 Agent 列表展示

每个项目下可以展开查看 Agent 列表。

Agent 展示内容包括：

- Agent 名称；
- 在线状态；
- 类型图标；
- 当前是否运行；
- 操作按钮。

Agent 状态包括：

状态	说明
在线	Agent 正常运行
离线	Agent 未启动
执行中	Agent 正在处理任务
异常	Agent 运行报错
暂停	Agent 被手动暂停

4.2.2 新增 Agent

用户可以在项目下新增 Agent。

新增 Agent 字段：

字段	必填	说明
Agent 名称	是	显示名称
Agent Key	是	系统唯一标识
所属项目	是	所属项目
Agent 类型	是	例如开发、测试、运维、数据分析
启动命令	否	例如 <code>npm start</code>
工作目录	否	Agent 的执行目录
环境变量	否	Agent 启动需要的环境变量
描述	否	Agent 能力说明

4.2.3 编辑 Agent

用户可以编辑 Agent 信息。

可编辑内容：

- Agent 名称；
- Agent 类型；
- 描述；
- 启动命令；
- 工作目录；
- 环境变量；
- 默认常用命令。

4.2.4 删除 Agent

用户可以删除 Agent。

删除规则：

- 删除前需要二次确认；
- 如果 Agent 正在运行，需要先停止或提示风险；
- 删除后，该 Agent 的对话记录和 Terminal 日志可以选择保留或归档。

4.2.5 Agent 切换

用户点击左侧 Agent 后：

- 中间对话区切换到该 Agent 的会话；
- 右侧 Terminal 区切换到该 Agent 的 TTY；

- 顶部显示当前项目和当前 Agent；
- 常用命令 Tag 根据当前 Agent 自动切换。

4.3 对话功能

4.3.1 消息展示

对话区需要展示用户与 Agent 的消息。

消息类型包括：

类型	说明
用户消息	用户输入的内容
Agent 消息	Agent 回复内容
系统消息	系统状态、错误、提醒
命令消息	通过快捷命令发送的内容
文件消息	上传的文件
代码消息	代码块、脚本内容

4.3.2 消息输入

输入框支持用户输入自然语言或命令。

基础能力：

- 输入文本；
- 回车发送；
- Shift + Enter 换行；
- 点击发送按钮发送；
- 支持粘贴代码；
- 支持上传附件；
- 支持插入常用命令。

4.3.3 消息发送

用户发送消息后：

1. 消息出现在对话区右侧；
2. 系统将消息发送给当前 Agent；
3. Agent 处理过程中显示「正在思考」或「执行中」状态；
4. Agent 返回内容后展示在对话区；
5. Terminal 区同步展示对应执行日志。

4.3.4 历史记录

系统需要保存每个 Agent 的对话历史。

要求：

- 每个 Agent 独立保存会话；
- 切换 Agent 后自动恢复历史消息；
- 支持清空当前 Agent 会话；
- 支持搜索历史消息；
- 支持按时间加载更多历史消息。

4.4 常用命令 Tag 功能

4.4.1 功能说明

在对话输入框的上方或下方，展示当前 Agent 的常用命令。

常用命令以 Tag 形式展示。

示例：

常用命令：

`/clear` `/compact` `/help` `/retry` `/summarize` + 添加命令

4.4.2 默认命令

系统默认提供以下常用命令：

命令	说明
<code>/clear</code>	清空当前上下文或当前会话
<code>/compact</code>	压缩当前上下文
<code>/help</code>	查看 Agent 支持的命令
<code>/retry</code>	重试上一次任务
<code>/summarize</code>	总结当前会话
<code>/status</code>	查看当前 Agent 状态

4.4.3 点击命令 Tag

用户点击某个命令 Tag 后，支持两种模式：

模式一：直接发送

- 点击 `/clear` 后立即发送该命令；
- 适合无参数命令。

模式二：填入输入框

- 点击命令后将命令插入输入框；
- 用户可以补充参数后再发送；
- 适合 `/run xxx`、`/deploy env=dev` 这类命令。

建议默认采用「填入输入框」模式，避免误操作。

4.4.4 新增常用命令

用户可以点击「+ 添加命令」新增常用命令。

新增字段：

字段	必填	说明
命令名称	是	例如 <code>/compact</code>
命令说明	否	鼠标悬停时展示
命令内容	是	实际发送内容
作用范围	是	当前 Agent / 当前项目 / 全局
点击行为	是	直接发送 / 插入输入框
排序	否	展示顺序

4.4.5 编辑常用命令

用户可以编辑已有命令。

可编辑内容：

- 命令名称；
- 命令内容；
- 命令说明；
- 作用范围；
- 点击行为；
- 排序。

4.4.6 删除常用命令

每个命令 Tag 右侧提供删除按钮。

删除规则：

- 用户点击删除后需要确认；
- 系统默认命令可以隐藏，但不建议彻底删除；
- 用户自定义命令可以彻底删除。

4.4.7 命令作用范围

常用命令支持三个作用范围：

范围	说明
全局命令	所有项目、所有 Agent 都展示
项目命令	仅当前项目下的 Agent 展示
Agent 命令	仅当前 Agent 展示

展示优先级：

Agent 命令 > 项目命令 > 全局命令

4.5 Node TTY Terminal 功能

4.5.1 Terminal 展示

右侧 Terminal 区用于展示当前 Agent 的运行终端。

内容包括：

- 启动命令；
- 运行日志；
- 错误日志；
- 当前进程状态；
- 当前目录；
- 命令输出。

4.5.2 Terminal 输入

第一阶段可以只做只读 Terminal。

第二阶段支持用户在 Terminal 中直接输入命令。

Terminal 输入能力：

- 输入 shell 命令；
- 回车执行；
- 支持快捷键；
- 支持复制粘贴；
- 支持历史命令；
- 支持清屏。

4.5.3 Terminal 操作按钮

Terminal 顶部提供操作按钮：

按钮	功能
清空	清空当前 Terminal 显示
复制	复制当前 Terminal 内容
放大	全屏查看 Terminal
重启	重启当前 Agent 进程
停止	停止当前 Agent
关闭	关闭当前 Terminal 面板

4.5.4 Terminal 与 Agent 绑定

每个 Agent 对应独立 Terminal。

切换 Agent 时：

- Terminal 自动切换；
- 保留该 Agent 的历史输出；
- 如果 Agent 未启动，显示未启动状态；
- 如果 Agent 异常，显示错误提示。

4.6 状态同步

系统需要同步展示当前 Agent 状态。

状态来源包括：

- Agent 服务状态；
- Node 进程状态；
- Terminal 连接状态；
- 最近一次任务执行状态；
- WebSocket 连接状态。

状态展示位置：

- 左侧 Agent 名称旁；
- 中间对话区顶部；
- 右侧 Terminal 顶部。

5. 权限需求

5.1 用户角色

系统至少支持以下角色：

角色	权限
普通用户	查看项目、使用 Agent、发送消息
项目管理员	管理当前项目、管理 Agent、管理项目命令
系统管理员	管理全部项目、全部 Agent、全局命令、用户权限

5.2 权限控制

需要控制以下操作权限：

- 新增项目；
- 编辑项目；
- 删除项目；
- 新增 Agent；
- 编辑 Agent；
- 删除 Agent；
- 新增常用命令；
- 删除常用命令；
- 执行 Terminal 命令；
- 停止 / 重启 Agent。

6. 数据模型设计

6.1 Project 项目表

字段	类型	说明
id	string	项目 ID
name	string	项目名称
key	string	项目标识
description	string	项目描述
icon	string	项目图标
sort	number	排序
status	string	状态
createdAt	datetime	创建时间
updatedAt	datetime	更新时间

6.2 Agent 表

字段	类型	说明
id	string	Agent ID
projectId	string	所属项目 ID

字段	类型	说明
name	string	Agent 名称
key	string	Agent 标识
type	string	Agent 类型
description	string	能力描述
status	string	在线状态
workdir	string	工作目录
startCommand	string	启动命令
env	json	环境变量
createdAt	datetime	创建时间
updatedAt	datetime	更新时间

6.3 Command 常用命令表

字段	类型	说明
id	string	命令 ID
name	string	命令名称，例如 <code>/clear</code>
content	string	实际发送内容
description	string	命令说明
scope	string	global / project / agent
projectId	string	项目 ID
agentId	string	Agent ID
actionType	string	send / insert
sort	number	排序
createdAt	datetime	创建时间
updatedAt	datetime	更新时间

6.4 Message 消息表

字段	类型	说明
id	string	消息 ID
projectId	string	项目 ID
agentId	string	Agent ID
role	string	user / agent / system

字段	类型	说明
content	text	消息内容
messageType	string	text / command / file / code
status	string	sending / success / failed
createdAt	datetime	创建时间

6.5 TerminalLog 终端日志表

字段	类型	说明
id	string	日志 ID
projectId	string	项目 ID
agentId	string	Agent ID
content	text	日志内容
level	string	info / warn / error
createdAt	datetime	创建时间

7. 接口需求

7.1 项目接口

GET	/api/projects	获取项目列表
POST	/api/projects	新增项目
PUT	/api/projects/:id	编辑项目
DELETE	/api/projects/:id	删除项目

7.2 Agent 接口

GET	/api/projects/:projectId/agents	获取项目下 Agent
POST	/api/projects/:projectId/agents	新增 Agent
PUT	/api/agents/:id	编辑 Agent
DELETE	/api/agents/:id	删除 Agent
POST	/api/agents/:id/start	启动 Agent
POST	/api/agents/:id/stop	停止 Agent
POST	/api/agents/:id/restart	重启 Agent
GET	/api/agents/:id/status	获取 Agent 状态

7.3 对话接口

GET	/api/agents/:agentId/messages	获取消息历史
POST	/api/agents/:agentId/messages	发送消息
DELETE	/api/agents/:agentId/messages	清空消息

7.4 常用命令接口

GET	/api/commands	获取命令列表
POST	/api/commands	新增命令
PUT	/api/commands/:id	编辑命令
DELETE	/api/commands/:id	删除命令

7.5 Terminal 接口

GET	/api/agents/:agentId/terminal/logs	获取终端日志
POST	/api/agents/:agentId/terminal/input	向终端发送命令
DELETE	/api/agents/:agentId/terminal/logs	清空终端日志

7.6 WebSocket

建议使用 WebSocket 推送以下实时数据：

/ws/agents/:agentId/chat	Agent 对话流式响应
/ws/agents/:agentId/terminal	Terminal 实时输出
/ws/agents/:agentId/status	Agent 状态变化

8. 交互细节

8.1 页面初始化

用户进入页面后：

1. 加载项目列表；
2. 默认选中最近使用的项目；
3. 默认选中最近使用的 Agent；
4. 加载该 Agent 的对话历史；
5. 加载该 Agent 的 Terminal 日志；
6. 建立 WebSocket 连接。

8.2 切换项目

用户点击项目时：

- 展开项目下的 Agent；

- 如果项目下有最近使用的 Agent，则自动选中；
- 如果没有 Agent，则提示创建 Agent。

8.3 切换 Agent

用户点击 Agent 时：

- 更新顶部面包屑；
- 加载对应对话；
- 加载对应 Terminal；
- 更新常用命令 Tag；
- 更新 Agent 状态。

8.4 发送常用命令

用户点击命令 Tag：

- 如果命令配置为「插入输入框」，则填入输入框；
- 如果命令配置为「直接发送」，则立即发送；
- 发送后该消息在对话区以命令消息形式展示。

8.5 新增命令

用户点击「+ 添加命令」：

1. 弹出命令配置窗口；
2. 输入命令名称和内容；
3. 选择作用范围；
4. 选择点击行为；
5. 保存后立即展示在常用命令区。

9. UI 设计要求

9.1 整体风格

建议采用深色科技风格。

关键词：

- 深色背景；
- 蓝色高亮；
- 卡片式布局；
- 三列分屏；
- 清晰的信息层级；
- 类似开发工具的专业感。

9.2 左侧菜单

要求：

- 项目可展开 / 收起；
- Agent 使用缩进展示；
- 当前选中 Agent 高亮；
- 在线 Agent 使用绿色状态点；
- 异常 Agent 使用红色状态点；
- 每个项目右侧支持新增 Agent、更多操作。

9.3 对话区

要求：

- 用户消息靠右；
- Agent 消息靠左；
- Agent 消息显示头像和名称；
- 支持代码块样式；
- 支持表格样式；
- 底部固定输入框；
- 常用命令 Tag 放在输入框上方或下方。

9.4 常用命令 Tag

要求：

- Tag 样式紧凑；
- 支持删除按钮；
- 支持 hover 显示说明；
- 支持拖拽排序；
- 「+ 添加命令」按钮与 Tag 保持同一行；
- 命令过多时自动换行或横向滚动。

9.5 Terminal 区

要求：

- 使用等宽字体；
 - 黑色背景；
 - 绿色命令提示符；
 - 日志内容清晰；
 - 支持滚动；
 - 支持全屏；
 - 支持复制。
-

10. 非功能需求

10.1 性能要求

- 页面首次加载时间不超过 3 秒；
- Agent 切换响应不超过 1 秒；
- 对话消息发送后 300ms 内展示在界面；
- Terminal 日志实时刷新延迟不超过 500ms；
- 单个 Agent 至少支持 1 万条历史日志分页加载。

10.2 安全要求

- 用户必须登录后访问；
- Terminal 命令执行需要权限控制；
- 敏感环境变量不可明文展示；
- 删除项目和 Agent 需要二次确认；
- 所有操作需要记录审计日志。

10.3 稳定性要求

- WebSocket 断开后自动重连；
- Agent 异常时要有明确提示；
- Terminal 输出过多时不能导致页面卡死；
- 对话发送失败时支持重试；
- 日志需要分页或增量加载。

10.4 可扩展性要求

后续需要支持：

- 多人协作；
- 多 Agent 任务编排；
- Agent 执行计划展示；
- 文件管理；
- Git 仓库绑定；
- 自动部署；
- 测试报告；
- 任务队列；
- 审批流；
- 飞书通知。

11. MVP 版本范围

第一阶段建议先实现 MVP，控制复杂度。

11.1 MVP 必做功能

- 三列布局；
- 项目列表展示；
- 项目新增、编辑、删除；

- Agent 列表展示；
- Agent 新增、编辑、删除；
- 点击 Agent 切换对话；
- 基础对话功能；
- 常用命令 Tag 展示；
- 常用命令新增、删除；
- Terminal 只读展示；
- Agent 状态展示。

11.2 MVP 暂不做功能

- 多人协作；
- Terminal 直接输入命令；
- 拖拽排序；
- 复杂权限系统；
- Agent 自动编排；
- 文件管理；
- Git 集成；
- 审批流。

12. 验收标准

12.1 项目管理验收

- 用户可以创建项目；
- 用户可以编辑项目名称；
- 用户可以删除项目；
- 项目可以在左侧菜单正确展示；
- 项目可以展开和收起。

12.2 Agent 管理验收

- 用户可以在项目下创建 Agent；
- 用户可以编辑 Agent；
- 用户可以删除 Agent；
- Agent 可以在项目下正确展示；
- 点击 Agent 后，中间对话区和右侧 Terminal 区能正确切换。

12.3 对话功能验收

- 用户可以发送消息；
- Agent 可以返回消息；
- 消息展示位置正确；
- 切换 Agent 后历史消息正确；
- 清空会话功能可用。

12.4 常用命令验收

- 输入框附近可以看到常用命令 Tag；
- 默认展示 `/clear`、`/compact`、`/help`；
- 用户可以新增命令；

- 用户可以删除命令；
- 点击命令后可以插入输入框或直接发送；
- 不同 Agent 可以展示不同命令。

12.5 Terminal 验收

- 右侧可以展示当前 Agent 的 Terminal 输出；
- 切换 Agent 后 Terminal 内容切换；
- Terminal 内容可以滚动；
- Terminal 可以清空；
- Terminal 可以复制内容。

13. 推荐技术方案

13.1 前端

建议技术栈：

- React
- TypeScript
- Tailwind CSS
- Zustand 或 Redux
- xterm.js
- WebSocket
- Vite

13.2 后端

建议技术栈：

- Node.js / NestJS
- WebSocket
- SQLite / PostgreSQL
- Prisma
- node-pty

13.3 Terminal 实现

推荐使用：

- 前端：xterm.js
- 后端：node-pty
- 通信：WebSocket

基础链路：

```
浏览器 xterm.js
  ↓ WebSocket
后端 Node 服务
```

↓ node-pty
真实 shell / Agent 进程

14. 后续迭代方向

14.1 Agent workflow编排

支持多个 Agent 串联执行任务，例如：

需求分析 Agent → 开发 Agent → Review Agent → 测试 Agent → 部署 Agent

14.2 项目任务看板

为每个项目增加任务视图：

- 待执行；
- 执行中；
- 待确认；
- 已完成；
- 失败。

14.3 版本发布联动

结合公司版本管理流程，支持：

- 提审检查；
- 更新前置条件校验；
- 自动生成发布清单；
- 事故复盘记录；
- 飞书通知。

14.4 权限与审计

增加完整的权限体系和操作审计：

- 谁创建了 Agent；
- 谁执行了命令；
- 谁删除了项目；
- 谁修改了环境变量；
- 哪个 Agent 执行失败。

15. 总结

Agent Console 是一个面向 AI Agent 管理和使用的三列式 Web 控制台。

核心结构为：

1. 左侧：项目 / Agent 菜单；
2. 中间：人与 Agent 的对话；
3. 右侧：Node TTY Terminal；
4. 输入框附近：常用命令 Tag，例如 `/clear`、`/compact`，并支持自定义增删。

第一阶段重点是把「项目管理、Agent 管理、对话、常用命令、Terminal 展示」跑通，先形成一个可用的 MVP。后续再逐步扩展到 Agent 编排、任务看板、自动部署、版本发布和权限审计。